

Computación Paralela y Distribuida

Ejercicio 3

Paralelismo en ciclos

El objetivo de este ejercicio es implementar paralelismo en ciclos utilizando la librería pcdp de la Universidad de Rice. El código entregado incluye un programa secuencial para multiplicar dos matrices. El resultado debe ser el código para ejecución en paralelo que se ejecute más rápido que la versión secuencial.

Las modificaciones solo se deben realizar en MatrixMultiply.java. No se pueden hacer cambios en las firmas (signatures) de los métodos públicos (public) ni en los protegidos (protected) dentro de MatrixMultiply, ni se pueden borrar. Se pueden adicionar métodos nuevos si se desea.

MatrixMultiply.java contiene varios PARA HACER:

Modificar el método MatrixMultiply.parMatrixMultiply para implementar la multiplicación de matrices en paralelo utilizando los métodos forall o forallChunked de pcdp. Hay un PARA HACER en MatrixMultiply.parMatrixMultiply para orientar los cambios que deben ser realizados. Una implementación paralela de MatrixMultiply.parMatrixMultiply debe mejorar la rapidez de forma casi lineal (i.e. la mejora en la rapidez lograda debe ser muy cercana al número de cores del computador).

Código ejemplo en las siguientes dos páginas

```

public static void seqMatrixMultiply(double[][] A, double B[], double[][] C, int n) throws
SuspendableException {
    long startTime = System.nanoTime();
    forseq(0, n-1, 0, n-1, (i,j) ->
        {
            C[i][j] = 0;
            for (int k =0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    );
    long timeInNanos = System.nanoTime() - startTime;
    printResults("seqMatrixMultiply", timeInNanos, C[n-1][n-1]);
}

```

```

public static void parMatrixMultiply(double[][] A, double B[], double[][] C, int n) throws
SuspendableException {
    long startTime = System.nanoTime();
    forall(0, n-1, 0, n-1, (i,j) ->
        {
            C[i][j] = 0;
            for (int k =0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    );
    long timeInNanos = System.nanoTime() - startTime;
    printResults("seqMatrixMultiply", timeInNanos, C[n-1][n-1]);
}

```

```

public static void parMatrixMultiplyChunked(double[][] A, double B[][], double[][] C, int n) throws
SuspendableException {
    long startTime = System.nanoTime();
    forallChunked(0, n-1, 0, n-1, (i,j) ->
        {
            C[i][j] = 0;
            for (int k =0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    );
    long timeInNanos = System.nanoTime() - startTime;
    printResults("seqMatrixMultiply", timeInNanos, C[n-1][n-1]);
}

private static void printResults(string name, long timeInNanos, double sum) {
    System.out.printf(" %s completed in %8.3f milliseconds, with C[n-1][n-1] = %8.5f \n", name,
timeInNanos / 1e6, sum);
}

```

Universidad Nacional de Colombia

Computación Paralela y Distribuida

Ejercicio 4

Sincronización de flujo de datos y Pipelining

En este ejercicio de deben aplicar phasers de java e hilos de java a un algoritmo de promedio iterativo uni-dimensional.

Descargar el archivo http://www.arcesio.net/paralela/ejercicio_4.zip

Las modificaciones solo se deben realizar en OneDimAveragingPhaser.java. No se pueden hacer cambios en las firmas (signatures) de los métodos públicos (public) ni en los protegidos (protected) dentro de OneDimAveragingPhaser, ni se pueden borrar. Se pueden adicionar métodos nuevos si se desea

OneDimAveragingPhaser.java contiene una actividad PARA HACER:

Basado en la version secuencial proporcionada en OneDimAveragingPhaser.runSequential y la version paralela en OneDimAveragingPhaser.runParallelBarrier, implementar una version paralela que utilice phasers para maximizar la superposición entre la terminación de la barrera y la realización de trabajo útil.

```

public static void seqMatrixMultiply(double[][] A, double B[], double[][] C, int n) throws
SuspendableException {
    long startTime = System.nanoTime();
    forseq(0, n-1, 0, n-1, (i,j) ->
        {
            C[i][j] = 0;
            for (int k =0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    );
    long timeInNanos = System.nanoTime() - startTime;
    printResults("seqMatrixMultiply", timeInNanos, C[n-1][n-1]);
}

```

```

public static void parMatrixMultiply(double[][] A, double B[], double[][] C, int n) throws
SuspendableException {
    long startTime = System.nanoTime();
    forall(0, n-1, 0, n-1, (i,j) ->
        {
            C[i][j] = 0;
            for (int k =0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    );
    long timeInNanos = System.nanoTime() - startTime;
    printResults("seqMatrixMultiply", timeInNanos, C[n-1][n-1]);
}

```

```

public static void parMatrixMultiplyChunked(double[][] A, double B[][], double[][] C, int n) throws
SuspendableException {
    long startTime = System.nanoTime();
    forallChunked(0, n-1, 0, n-1, (i,j) ->
        {
            C[i][j] = 0;
            for (int k =0; k < n; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    );
    long timeInNanos = System.nanoTime() - startTime;
    printResults("seqMatrixMultiply", timeInNanos, C[n-1][n-1]);
}

private static void printResults(string name, long timeInNanos, double sum) {
    System.out.printf(" %s completed in %8.3f milliseconds, with C[n-1][n-1] = %8.5f \n", name,
timeInNanos / 1e6, sum);
}

```